Design Write-Up

Live Data AI Assistant — Telegram Bot (n8n + Apify + Gemini)

Architecture Overview

The assistant runs entirely in n8n, triggered by a Telegram bot. Every message first passes through a Gemini classifier that returns a structured route (LIVE or GENERAL) and category in one call, driving a visible IF/ELSE branch — no keyword matching involved. LIVE queries go to an Apify Google Search Scraper actor; raw results are reshaped into clean structured JSON (title, description, source, URL, date) before Gemini summarizes them into a bullet-formatted answer. GENERAL queries skip Apify and go straight to Gemini. Both paths converge into one shared Telegram reply node and one shared Google Sheets logging node, so every interaction is always answered and always logged with a timestamp, category, route, summary, response, and status.

Design Decisions

Single-call structured classification returns route and category together as one JSON object, halving classification API usage versus two separate calls. **Structured data before summarization** means Apify's results are always reshaped into a fixed schema before Gemini sees them, never raw scrape output. **Shared convergence points** — both branches and all six error paths feed into the same two terminal nodes via `$json` rather than hardcoded node names, avoiding duplicated logic. **Graceful degradation everywhere** — the Apify call, the empty-results case, and each of the three Gemini calls has its own error branch with a distinct fallback message, so one failed call never leaves the user without a reply.

Challenges Faced

Exposing local n8n to the internet was the first hurdle — Telegram's webhook needs public HTTPS, which `localhost` can't provide. n8n's built-in tunnel mode was unreliable, so the workflow moved to ngrok with a reserved static domain, avoiding the broken-webhook/OAuth-redirect cycle caused by a changing URL on every restart.

A trickier bug came from n8n's expression scoping: `$json` reflects the immediately preceding node, not the original trigger data. Inserting a Telegram "typing indicator" node before the classifier silently broke its access to the user's query, since it read the typing-indicator API's own response instead — causing inconsistent misclassifications, fixed by explicitly referencing `$('Extract User Query').item.json`. The same cause appeared in the Sheets logging node, where a column referencing one branch's specific node failed whenever the other branch ran instead — resolved with the ambient `$json`, correct precisely because that node receives input from multiple possible upstream branches.

Telegram also rejected replies containing unmatched markdown from Gemini's output, fixed by stripping *, _, and backticks before sending, while prompting Gemini to use a plain bullet character instead. Finally, Gemini's free-tier rate limits were hit repeatedly during testing — a genuine failure that validated the error-handling design directly, with the fallback path replying gracefully instead of leaving the user without a response.